

KalaSetu — Team Knowledge & Defense Guide

SIH THEME: HERITAGE & CULTURE

Plain-English Architecture, Codebase Truths & Judge Q&A Defense | Smart India Hackathon

Section 1: What This App Is, In One Paragraph

Imagine you are an incredible rural woodcarver or saree weaver in a village. You create world-class art with your hands, but you cannot sell it online because you find English e-commerce forms with barcodes, dimensions, and descriptions intimidating or impossible to navigate. Because of that, predatory middlemen buy your ₹3,000 saree for ₹600 and pocket the rest. **KalaSetu** is a mobile app that removes that barrier completely: the artisan just taps a single microphone button, speaks naturally in Hindi or English (for example: *"I made a handloom silk saree, it took me 3 days, I want ₹3,500 for it"*), or snaps a quick photo. Our artificial intelligence instantly listens, analyzes the craft, writes a polished product listing, and puts it online with a transparent "Fair Share" meter showing buyers that 95% of their money goes directly into the artisan's pocket. It also connects the artisan to government welfare schemes like PM Vishwakarma and protects them from machine-made fakes with scannable physical QR certificates.

Section 2: The Full Tech Stack — What We Used and WHY

Every tool below is actually implemented in the codebase right now. Here is what each does and why we chose it:

1. React (Frontend Library)

UI Construction

What it is: A popular tool created by Facebook that lets programmers build user interfaces out of reusable Lego-like building blocks called "components."

Its job in KalaSetu: Every screen you see—the product cards, the login screen, navigation bars, and the Fair Share meter—is built as a React component.

Why we chose it: Instead of writing dozens of separate web pages, React updates only the specific part of the screen that changes (like incrementing the cart count) without flickering or reloading the entire page.

2. Tailwind CSS (Styling Framework)

Design System

What it is: A styling toolkit that lets developers style buttons, colors, and layouts directly in the code using pre-made visual classes.

Its job in KalaSetu: It provides our visual theme—heritage navy blue (`#003366`), warm terracotta orange (`#FF9933`), rounded corners, and responsive spacing that looks great on all mobile screens.

Why we chose it: Traditional CSS requires maintaining large separate style sheets. Tailwind allows instant, standardized styling without bloated code, ensuring fast load times on budget devices.

3. Firebase Authentication (User Login System)

Security & Identity

What it is: A cloud security service managed by Google that handles user verification and sign-in.

Its job in KalaSetu: It runs our real Phone Number + SMS OTP (One-Time Password) login for both Buyers (Citizens) and Artisans.

Why we chose it: Rural artisans rarely use emails or remember passwords. Building a custom SMS gateway and OTP verification server from scratch is time-consuming, expensive, and insecure. Firebase provides reliable, carrier-grade SMS delivery with invisible bot protection out of the box.

4. Firebase Firestore (Cloud Database)

Real-Time Cloud DB

What it is: A cloud database by Google that stores data in structured digital folders called "collections" and "documents."

Its job in KalaSetu: It stores our 12 curated Indian handicraft items, artisan profiles, individual user shopping carts, and completed orders.

Why we chose it: Traditional databases require managing physical servers. Firestore connects directly to the frontend and features **real-time listeners**—when an artisan publishes an item, it appears on the buyer's screen instantly without refreshing.

5. Web Speech API (Voice Recognition)

Client-Side Voice

What it is: A built-in browser engine (in Chrome, Edge, Safari) that translates spoken voice into written text in real time.

Its job in KalaSetu: When an artisan taps "Speak to AI", this browser engine listens through their microphone and converts their spoken Hindi or English into text.

Why we chose it: It requires **no expensive third-party speech servers**, has zero latency, works directly inside the browser, and natively supports Indian language accents and Hindi (`hi-IN`).

6. Google Gemini 3.7 Flash (Generative Artificial Intelligence)

Multimodal AI Engine

What it is: Google's latest high-speed artificial intelligence model capable of understanding both text and images simultaneously.

Its job in KalaSetu: It takes the raw voice transcript or craft photograph and automatically extracts a professional title, culturally authentic description, category, tags, and suggested price.

Why we chose it: Unlike older AI models that return conversational text that can break app code, Gemini supports **Strict JSON Schema Output**. It guarantees answers in the exact format our database expects. It is also fast (under 2 seconds) and cost-effective.

7. Progressive Web App (PWA) via `vite-plugin-pwa`

Offline & Installability

What it is: Web technology that allows a website to be installed on a mobile home screen like a Play Store app and work offline.

Its job in KalaSetu: It caches the app's visual files and icons on the phone so the app opens instantly on spotty 2G/3G rural networks.

Why we chose it: Artisans with budget phones cannot afford 100MB Play Store downloads. A PWA installs in seconds, takes less than 3MB of phone storage, and updates automatically.

8. Capacitor (Native Android APK Builder)

Mobile Runtime

What it is: A cross-platform runtime tool created by Ionic that wraps web code into an official native Android Studio project.

Its job in KalaSetu: It compiles our React application into a standalone `app-debug.apk` that can be installed on any physical Android phone for the live demo.

Why we chose it: Writing separate apps in Flutter or Kotlin would require managing two codebases. Capacitor lets us build one polished web codebase that works everywhere.

Section 3: How Each Major Feature Actually Works, Step by Step

1. Voice-to-Catalog AI

1. **User Action:** The artisan taps "Speak to AI" on the Add Item screen.
2. **Language Selection:** The app checks active language; if Hindi, it sets `recognition.lang = 'hi-IN'`; otherwise `'en-US'`.
3. **Listening & Local Transcription:** The microphone records audio waves and transcribes speech to text locally in the browser.
4. **Displaying Transcript:** When speaking stops, the app switches to an "AI is analyzing..." loading screen and quotes the transcribed text on-screen so the artisan knows the mic heard them clearly.
5. **Gemini AI Formatting:** The text is sent to Google's `gemini-3.7-flash` model with a strict JSON schema requiring: `title`, `description`, `category`, `tags`, and `price`.
6. **Form Population:** Gemini responds with structured JSON in under 2 seconds. The app sets the suggested price, computes the 5% platform fee (`finalPrice = Math.round(price * 1.05)`), and populates all form fields.
7. **Artisan Verification:** The artisan reviews the pre-filled form, adjusts any text or price if desired, and clicks "Publish to Catalog."

2. Photo-to-Listing AI

1. **User Action:** The artisan taps the camera card, opening their phone's native camera or gallery.
2. **Base64 Conversion:** The selected image is read into memory using the browser's `FileReader` API.
3. **Prefix Stripping:** The code strips the `data:image/jpeg;base64,` header to isolate raw binary Base64 characters required by the Gemini Vision API.
4. **Visual Analysis:** The Base64 image is sent to Gemini 3.7 Flash with a prompt asking the model to identify the handicraft, draft an authentic story, categorize it, and estimate a fair starting price.
5. **Verification Form:** The form loads with the photo preview and the AI-generated details ready for artisan review.

3. Fair Pricing Calculation & Transparency Meter

The Exact Formula:

- `Buyer Price (Final Price) = Artisan's Suggested Price * 1.05`
- `Platform Fee = Buyer Price - Artisan's Suggested Price (5% operational fee)`
- `Artisan Share Percentage = Round((Artisan Ask / Buyer Price) * 100) = 95%`

In the UI: On the Product Details page, buyers see an interactive dual-color progress bar: green represents the **95% Artisan Share (₹1,000)** and blue represents the **5% Platform Fee (₹50)**. This transparency builds buyer trust and proves the artisan is not being exploited.

4. QR Code Generation & The Artisan Story Page

1. **Unique Digital Address:** Each artisan has an authenticated page: `{origin}/artisan/{artisan.id}`.
2. **SVG Generation:** The app renders an SVG QR code using `qrcode.react` with High Error Correction (`level="H"`), meaning it remains scannable even if 30% of a printed label is smudged.
3. **Physical-to-Digital Link:** Artisans can attach this QR tag to physical packaging as an official authenticity badge.
4. **Story Verification:** Scanning with any smartphone camera opens the artisan's public profile, showing their craft lineage, state of origin, official registration ID (`IN-KALA-[ID]`), and verified product catalog.

5. The Phone OTP Login Flow

1. **Mobile Entry:** The user enters a standard 10-digit Indian phone number.
2. **Invisible Security Check:** Firebase initializes an invisible reCAPTCHA verifier to prevent bot abuse without showing puzzle captchas.
3. **SMS Dispatch:** Google's telecom infrastructure sends a real 6-digit SMS verification code to the phone.
4. **Verification:** The user inputs the 6 digits, and Firebase confirms the session, issuing a secure auth token stored in browser IndexedDB.
5. **Role Routing:** Buyers go straight to the Discover feed. Artisans fill out their name, craft, and region to create an official profile in Firestore.

6. The Hindi Language Switcher (i18n)

1. **Central Dictionary:** All UI strings are stored in `src/lib/translations.ts` with parallel keys for English (`en`) and Hindi (`hi`).
2. **Persistent Choice:** The active language is saved to `localStorage` so it persists across page reloads.
3. **Instant Toggle:** Clicking the Globe icon flips the React state. The entire interface changes to Hindi instantly without network delays.
4. **Voice Adaptation:** Crucially, switching to Hindi sets the Web Speech API recognition language code to `hi-IN`, ensuring the AI correctly transcribes spoken Hindi.

7. Offline–First Sync Architecture

1. **Connectivity Detection:** `AppContext.tsx` monitors browser `online` and `offline` events.
2. **Local Fallback:** If an artisan adds a craft while offline, the item receives a local ID, is tagged with `syncStatus: 'pending'`, and is stored in state.
3. **Visual Indicator:** In "My Catalog", the product shows an orange badge: *"Sync Pending (Offline)"*.
4. **Auto Cloud Sync:** Once internet is restored, an automated background effect detects pending items and updates them to Firebase Firestore with status *"synced"*.

Section 4: "Why Did You Build It This Way?" — Anticipated Judge Q&A

Q: Why did you choose React + Vite instead of Flutter or React Native?

"Our core audience is marginalized rural artisans with budget phones and limited storage. Flutter and React Native apps require downloading a 40–80MB APK from the Play Store. By using React + Vite configured as a PWA and Capacitor, our app loads instantly over the web in under 2MB, installs to the home screen in seconds, works offline, and still compiles to a native Android APK when needed."

Q: How does the AI generate the price suggestion — is it random or calculated?

"It is strictly contextual. Gemini first searches the spoken transcript for an explicit price mentioned by the artisan (e.g., 'I want ₹500'). If none is mentioned, it estimates a fair baseline based on craft category and labor complexity. Crucially, the AI never forces a price: the artisan retains 100% control on an editable confirmation screen before anything is published."

Q: Is the voice recognition happening on-device or sent to a server?

"Transcription is handled directly by the browser's native Web Speech API using local operating system models with zero latency. Only the resulting text transcript is sent to Gemini for structuring. This saves significant bandwidth, which is essential for rural 2G/3G connectivity."

Q: How is user data stored and is it secure?

"User authentication tokens are encrypted in browser IndexedDB via Firebase Auth. In Firestore, cart data is partitioned under `carts/{user.uid}`, preventing users from accessing each other's data. We also deliberately avoid storing sensitive personal identifiers or bank details in the database."

Q: What parts of this prototype are fully functional vs. simulated?

"100% Real & Live: Phone OTP login via Firebase Auth, real-time Firestore catalog, cart, and orders, live Web Speech voice recognition in English and Hindi, Gemini 3.7 Flash text & vision extraction, Hindi language toggle, and native Android APK.

Simulated for Hackathon: Checkout payment is simulated (no live merchant Razorpay key), government scheme cards show real guidelines but link out to official portals, and the WhatsApp bot is demonstrated as an interactive UI walkthrough."

Q: Is this app legally using any government emblems or logos?

"No. Under the State Emblem of India Act, private hackathon apps cannot legally use the Lion Capital of Ashoka or official ministry seals. We created our own brand mark—KalaSetu—and used a generic tricolor ribbon accent to celebrate heritage respectfully without violating statutory emblem laws."

Section 5: Simple Glossary

API

A digital bridge allowing two pieces of software to send data to each other.

Frontend

Everything the user sees and clicks on their screen.

Backend

The behind-the-scenes databases and servers that store and manage app data.

Component

A reusable Lego-like building block of a user interface (like a button or card).

Firestore

Google's cloud database that syncs data across users in real time without refreshing.

PWA

A website that can install onto a phone home screen and work like a real app.

Service Worker

A background script that saves app files on the phone so it works offline.

LLM (AI)

An advanced AI program (like Gemini) trained to understand and generate human language.

JSON

A clean computer format for organizing data using key-value labels.

Base64

A way to convert pictures into text code so they can be sent across the web.

Capacitor

A tool that packages web code into a native Android APK application.

GI Tag

An official government certification for authentic regional crafts (e.g. Kanchipuram Silk).

Section 6: Quick-Reference Cheat Sheet (5-Minute Review)

The 3 Golden Rules to State with Confidence:

1. **The Problem We Solve:** *"We remove the digital literacy barrier for rural artisans by replacing complicated text forms with simple voice and photo AI."*
2. **The Economic Difference:** *"Unlike big marketplaces that deduct 20–30% in fees, our Fair Share Meter guarantees that 95% of every rupee goes straight to the artisan."*
3. **Authenticity:** *"Every product has a scannable QR certificate linked to the artisan's verified government registration and GI status to fight counterfeit goods."*

Layer	Technology	Primary Job in KalaSetu
Frontend	React 19 + Vite	Fast, responsive mobile component interface
Styling	Tailwind CSS v4	Indian heritage design system (navy blue & terracotta)
Authentication	Firebase Auth	Real Phone number + SMS OTP login
Database	Firebase Firestore	Real-time cloud database syncing catalog, cart & orders
Voice Input	Web Speech API	Zero-cost, local voice-to-text in English & Hindi
AI Model	Google Gemini 3.7 Flash	Extracts structured title, story & price from voice/images
Packaging	Capacitor + PWA	Dual delivery: lightweight web PWA & native Android APK

90-Second Demo Order: Discover Feed → Product Details (Fair Share Meter) → Artisan Story (QR Certificate) → Switch to Artisan Mode → Voice AI Add Item → Language Toggle (Hindi) → PM Vishwakarma Schemes.